



CODEGEN PLAYBOOK

Версия: 1.0.0

Дата: 2026-06-13

Статус: Active

Автор: Koda (NLP-Core-Team)



НАЗНАЧЕНИЕ

Этот playbook описывает workflow кодогенерации с использованием SUMMARY_DOCS.

Primary Purpose: Обеспечить консистентный codegen процесс с documentation-first подходом.



CODEGEN WORKFLOW

Шаг 1: Load Context

1. Прочитать SUMMARY_DOCS/INDEX.md
 - Понять структуру документации
2. Прочитать SUMMARY_DOCS/MANIFEST.json
 - Получить список документов
 - Найти relevant contracts
3. Прочитать SUMMARY_DOCS/appendix/AI_ENTRYPOINTS.md
 - Следовать AI entry flow
4. Прочитать SUMMARY_DOCS/adr/ADR_INDEX.md
 - Понять архитектурные решения
5. Прочитать SUMMARY_DOCS/appendix/domain-glossary.md
 - Понять терминологию

Шаг 2: Load Node Tree Context

1. Прочитать SUMMARY_DOCS/nodes/NODETREE_INDEX.md
 - Понять дерево узлов
 - Определить ветки (production/alpha/working)
2. Прочитать SUMMARY_DOCS/nodes/NODETREE_MANIFEST.json
 - Получить machine-readable node registry
 - Найти priority 1 technical nodes
3. Прочитать relevant node contracts:

- BranchNodeContract.md
- DomainNodeContract.md
- TechnicalNodeContract.md
- NODE_CONTRACT_<node-id>.md

4. Извлечь specifications:

- Node identity и branch binding
- Domain rules (dev without domains, prod with domains)
- Technical nodes priority
- Settings surface

Шаг 3: Load State Files

1. Прочитать node state files:

- SUMMARY_DOCS/state/branch-tree.json (ветки)
- SUMMARY_DOCS/state/domain-tree.json (домены)
- SUMMARY_DOCS/state/node-settings-map.json (настройки)
- SUMMARY_DOCS/state/node-runtime-map.json (runtime mapping)
- SUMMARY_DOCS/state/node-codegen-map.json (codegen relevance)
- SUMMARY_DOCS/state/node-priority-map.json (приоритеты)

2. Извлечь configuration:

- Node configurations
- Domain mappings
- Settings inheritance
- Runtime targets (localhost vs domains)

Шаг 4: Load Node Models

1. Прочитать node model docs:

- SUMMARY_DOCS/nodes/NODE_SETTINGS_MODEL.md (уровни настроек)
- SUMMARY_DOCS/nodes/NODE_RUNTIME_MODEL.md (runtime behavior)
- SUMMARY_DOCS/nodes/NODE_CODEGEN_MODEL.md (codegen process)

2. Извлечь models:

- Settings hierarchy (project-global → runtime-local)
- Environment modes (local/dev, working, alpha, production)
- Codegen operations (create, update, local config, prod config)

Шаг 4.5: Load Node Matrices

1. Прочитать node matrices:

- SUMMARY_DOCS/nodes/NODE_CAPABILITY_MATRIX.md (capabilities by version)
- SUMMARY_DOCS/nodes/NODE_ACCESS_MATRIX.md (access/security rules)
- SUMMARY_DOCS/nodes/NODE_DEPENDENCY_MATRIX.md (node dependencies)

- SUMMARY_DOCS/nodes/NODE_FAMILIES.md (node families)

2. Извлечь matrices:

- Version-scoped capabilities (3.0.*, 3.1.*, 4.*)
- Access levels (public/private/restricted/internal)
- Dependencies (hard/optional/planned/environment-specific)
- Family groupings (api family, storage family, etc.)

Шаг 5: Load Topology

1. Прочитать topology docs:

- NETWORK_MAP.md (сетевая топология)
- DEPLOYMENT_MAP.md (deployment карта)
- DOMAIN_MAP.md (доменная карта)

2. Извлечь architecture:

- Network paths
- Deployment targets
- Domain routing

Шаг 6: Generate Code

1. Использовать node contracts как specifications
2. Использовать state files как configuration
3. Использовать node models как architecture
4. Использовать topology как infrastructure
5. Генерировать код следуя contracts
6. Проверить соответствие specifications

Codegen Rules:

- dev mode MUST NOT require real domains
- prod mode MUST use canonical production domains
- working MAY run via localhost
- Technical nodes (priority 1) first
- Preserve production identity in all modes

Шаг 7: Verify

1. Проверить соответствие node contracts:

- Node ID совпадает?
- Branch binding верный?
- Domain rules соблюдены (dev without domains, prod with domains)?
- Technical nodes priority 1?

2. Проверить соответствие state:
 - Configuration актуальна?
 - Runtime mapping верный?
 - Settings inheritance correct?
3. Проверить соответствие models:
 - Settings hierarchy respected?
 - Environment mode correct?
 - Codegen operations valid?
4. Проверить соответствие topology:
 - Network paths корректны?
 - Deployment targets верны?

Шаг 8: Update Documentation

Если код изменился:

1. Обновить relevant node contracts (если specs изменились)
2. Обновить node state files (если configuration изменилась)
3. Обновить topology docs (если architecture изменилась)
4. Обновить NODETREE_MANIFEST.json (если добавлены узлы)
5. Обновить SUMMARY_DOCS/MANIFEST.json (если добавлены документы)
6. Обновить doc-state.json (metadata)

Шаг 9: Commit

Commit message format:

[CODEGEN]

Node Contracts:

State:

Files:

Docs:

Example:

[CODEGEN] Update node deployment scripts

Contracts: NodeDeploymentContract.md, NodeTreeContract.md

State: node-tree.json

Files: deploy.sh, docker-compose.yml

Docs: DEPLOYMENT_MAP.md

CONTEXT PACKAGE

Minimal Context:

```
{
  "entry_point": "SUMMARY_DOCS/INDEX.md",
  "manifest": "SUMMARY_DOCS/MANIFEST.json",
  "contracts": [
    "SUMMARY_DOCS/contracts/node-contracts/NodeTreeContract.md"
  ],
  "state": [
    "SUMMARY_DOCS/state/node-tree.json"
  ]
}
```

Standard Context (Node Codegen):

```
{
  "entry_point": "SUMMARY_DOCS/INDEX.md",
  "manifest": "SUMMARY_DOCS/MANIFEST.json",
  "adr": {
    "index": "SUMMARY_DOCS/adr/ADR_INDEX.md",
    "adr001": "SUMMARY_DOCS/adr/ADR-001-branch-node-model.md",
    "adr002": "SUMMARY_DOCS/adr/ADR-002-dev-without-domains-prod-with-
domains.md",
    "adr003": "SUMMARY_DOCS/adr/ADR-003-technical-nodes-first.md",
    "adr004": "SUMMARY_DOCS/adr/ADR-004-summary-docs-as-node-source-of-
truth.md",
    "adr005": "SUMMARY_DOCS/adr/ADR-005-working-alpha-production-release-
flow.md"
  },
  "nodes": {
    "index": "SUMMARY_DOCS/nodes/NODETREE_INDEX.md",
    "manifest": "SUMMARY_DOCS/nodes/NODETREE_MANIFEST.json",
    "contracts": [
      "SUMMARY_DOCS/contracts/nodes/BranchNodeContract.md",
      "SUMMARY_DOCS/contracts/nodes/DomainNodeContract.md",
      "SUMMARY_DOCS/contracts/nodes/TechnicalNodeContract.md",
      "SUMMARY_DOCS/contracts/nodes/NODE_CONTRACT_<node-id>.md"
    ]
  }
}
```

```

    ],
    "matrices": [
        "SUMMARY_DOCS/nodes/NODE_CAPABILITY_MATRIX.md",
        "SUMMARY_DOCS/nodes/NODE_ACCESS_MATRIX.md",
        "SUMMARY_DOCS/nodes/NODE_DEPENDENCY_MATRIX.md"
    ]
},
"state": [
    "SUMMARY_DOCS/state/branch-tree.json",
    "SUMMARY_DOCS/state/domain-tree.json",
    "SUMMARY_DOCS/state/node-settings-map.json",
    "SUMMARY_DOCS/state/node-runtime-map.json",
    "SUMMARY_DOCS/state/node-codegen-map.json",
    "SUMMARY_DOCS/state/node-priority-map.json",
    "SUMMARY_DOCS/state/node-capability-map.json",
    "SUMMARY_DOCS/state/node-access-map.json",
    "SUMMARY_DOCS/state/node-dependency-map.json",
    "SUMMARY_DOCS/state/node-doc-health.json"
],
"models": [
    "SUMMARY_DOCS/nodes/NODE_SETTINGS_MODEL.md",
    "SUMMARY_DOCS/nodes/NODE_RUNTIME_MODEL.md",
    "SUMMARY_DOCS/nodes/NODE_CODEGEN_MODEL.md"
],
"glossary": [
    "SUMMARY_DOCS/appendix/domain-glossary.md",
    "SUMMARY_DOCS/appendix/entity-definitions.md"
]
}

```

CHECKLISTS

Pre-Codegen Checklist:

- ☐ INDEX.md прочитан
- ☐ MANIFEST.json загружен
- ☐ Relevant contracts найдены
- ☐ State files загружены
- ☐ Topology docs прочитаны
- ☐ Context package собран

Post-Codegen Checklist:

- ☐ Код соответствует contracts
- ☐ Код соответствует state
- ☐ Код соответствует topology
- ☐ Документация обновлена
- ☐ MANIFEST.json обновлён
- ☐ doc-state.json обновлён

- ☐ Commit message корректен

VERIFICATION

Contract Compliance:

```
# Проверить что код использует correct node names
grep -r "work_server" code/
grep -r "home_nas" code/

# Проверить что код следует contracts
node scripts/verify-contracts.js
```

State Compliance:

```
# Проверить что код использует correct configuration
node scripts/verify-state.js
```

Documentation Updates:

```
# Проверить что документация актуальна
node scripts/check-doc-links.js

# Валидировать MANIFEST
node scripts/validate-manifest.js
```

BEST PRACTICES

Do:

- ☒ Всегда читать contracts перед генерацией
- ☒ Проверять соответствие specifications
- ☒ Обновлять документацию при изменениях
- ☒ Использовать canonical paths
- ☒ Следовать policies

Don't:

- ☒ Не генерировать код без чтения contracts
- ☒ Не игнорировать state files
- ☒ Не забывать обновлять документацию
- ☒ Не использовать legacy paths

- **X** Не пропускать verification

RELATED DOCUMENTS

- [INDEX.md](#) — Главная навигация
- [MANIFEST.json](#) — Индекс документов
- [AI_ENTRYPOINTS.md](#) — AI workflow
- [DOC_CODEGEN_POLICY.md](#) — Политика кодогенерации
- [doc-update-playbook.md](#) — Обновление документации

Создано: 2026-06-13

Версия: 1.0.0

Статус: Active

Автор: Koda (NLP-Core-Team)

 **Balloo - Переверни общение!**